FD E-COMMERCE ANALYTICS DASHBOARD

INTERDISCIPLINARY MINOR PROJECT — REVISED IMPLEMENTATION REPORT

Submitted by: Jagjot Singh Passi
Roll no.: 2446571
Programme: Bachelor of Computer Applications (BCA)
Institute: Punjab College of Technical Education (PCTE), Ludhiana
Course instructor: Ms. Sarbjeet Kaur
Implementation version: FD Intelligence v2.0 (educational prototype)
Reporting date: October 2026
Data source: Clearly labeled, realistic *synthetic* demonstration data unless actual store data are separately imported and authorized.

ABSTRACT

The FD E-Commerce Analytics Dashboard is an interactive business intelligence system designed to consolidate product, customer, transaction, order-item and advertising records into an interpretable set of e-commerce performance indicators. The application's primary functions are to describe **sales trends**, analyze **customer preferences and repeat purchase behavior**, and evaluate **marketing return on investment** (with explicit definitions and attribution assumptions). It assists an FD e-commerce manager in comparing products, customer regions, time periods and advertising channels for data-driven decisions. The implemented technical stack is Python, Streamlit, Pandas, Plotly and MongoDB, with SQLite provided as an offline demonstration mode. Data arrive through validated Excel/CSV uploads or a transparent reproducible synthetic generator. The system includes data-lineage labels, inter-table referential integrity validation, export and backup facilities, an administrative change log and a faculty demonstration guide.

Scope boundary: The current application is not a full checkout e-commerce website. It is an analytics dashboard **for** an e-commerce store. It does not ingest live orders from a commercial platform, and included demonstration data are not real FD-store sales.

---

1. INTRODUCTION

A typical small online seller records orders and product information in different spreadsheets. Advertising platforms independently report spend, clicks and impressions. Customer information may also exist in a separate CRM export. Without integration, comparing revenue with campaign costs or measuring repeat buyers becomes slow and error-prone. This minor project addresses the analytics side of that issue by defining six consistent data entities, joining them for commercial analysis and presenting results in an interactive web dashboard.

1.1 Official project title and faculty brief

Title: FD E-Commerce Analytics Dashboard.

Faculty description (supplied with the topic): Create a digital dashboard to analyze sales trends, customer preferences, and marketing ROI for a FD e-commerce store; build interactive visualizations and data integrations that support strategic decisions.

1.2 Problem statement

An FD store manager cannot reliably decide which category to invest in, which customers may stop buying or which advertising channel contributes the most sales when those sources are disconnected and analysis is manually performed in spreadsheets.

1.3 Project proposal vs actual technology

The original six-page synopsis used **MERN: MongoDB, Express, React and Node**, while the Phase 1 evaluation sheet recorded **Python, MongoDB and Streamlit**. The originally uploaded application ran against **SQLite** (decorline.db). The revised implementation selects **Python** + **MongoDB** + **Streamlit** to match the Phase 1 evaluated technology. SQLite remains a distinctly named local fallback for demonstration without MongoDB. The MERN text in the earlier synopsis must be formally updated; do not claim the final implementation uses React/Express.

---

2. RATIONALE AND SIGNIFICANCE

- **Reduced reporting fragmentation:** standard schemas and validation checks reduce manual reconciliation work.
- **Visibility into sales performance:** completed orders can be analyzed by time, region, product and category.
- **Visibility into customer behavior:** repeat buying, recency and value segmentation help target retention strategies.
- **Campaign accountability:** marketing expense can be compared with explicitly attributed orders.
- **Educational benefit:** the system connects database modeling, Python analytics, data quality, interactive visualizations, and marketing concepts.
- **Transparent evidence:** source labels and analytical limitations reduce the risk of presenting simulated records as actual company financial results.

A dashboard improves the presentation of information, not necessarily the validity of its inputs. The validation, lineage and formula definitions are therefore essential parts of the project, not decorative additions.

---

3. OBJECTIVES

1. Design a centralized, browser-based FD e-commerce analytics dashboard.
2. Maintain structured records for products, customers, orders, order lines, campaigns and order attribution.
3. Allow database-backed data persistence using MongoDB, with an optional SQLite offline demo.
4. Import, validate and reconcile Excel or CSV source files without destroying active data after a failed validation.
5. Calculate correct realized sales indicators and visualize product/category, geographic, seasonal and payment patterns.
6. Analyze customer purchasing behavior using RFM, first-purchase cohorts, repeat rate, and exploratory clustering.
7. Compare campaign spend and order attribution using ROAS, CPA and an explicitly limited contribution ROI proxy.
8. Provide interactive filters, CSV/Excel exports, backup facilities, audit logging and rule-driven management suggestions.
9. Demonstrate operation and limitations in a repeatable faculty evaluation using automated tests.

---

4. LITERATURE REVIEW / BACKGROUND TECHNOLOGIES

4.1 Business intelligence dashboards

Traditional spreadsheets enable flexible ad hoc reporting but often spread formula logic across files. Purpose-built interactive dashboards such as the open-source concepts used by Metabase and Redash, or established BI services such as Power BI and Tableau, organize KPIs into consistent visual comparisons. This prototype emphasizes

transparent and inspectable formulas tailored to a narrowly defined educational use case rather than competing as a general enterprise BI platform.

4.2 Web development platform

Streamlit renders Python-driven interactive apps into a browser; a user can select filters without writing JavaScript. Python data frames and Plotly charts permit detailed analyses with relatively few dependent layers. This differs from the planned MERN architecture, where a separate React frontend would call Express APIs.

4.3 Databases

MongoDB stores related entities as collections and offers persistent records with an approachable document API (pymongo). The active dataset is identified by a batch ID so an import does not expose only some of the six collections. **SQLite** is an embedded local demonstration store. SQLite provides an easy portable mode but should not be mislabeled MongoDB.

4.4 Customer analytics: RFM

RFM stands for *recency*, *frequency* and *monetary value*. Recency counts days since the last delivered order, frequency counts delivered orders and monetary value sums delivered order value. Quintile-style rank scores classify customers into interpretable groups such as Champions, Loyal and At risk. RFM is descriptive segmentation, not proof that any particular person will purchase again.

4.5 Marketing analytics

Revenue attribution associates particular completed orders with a campaign under an explicit tracking model. This implementation uses a **single-touch** relationship: an order may have at most one campaign attribution. ROAS and CPA describe that mapping, while contribution ROI includes estimated product cost. They cannot establish a campaign's *incremental* causal impact without an experimental design or other identification strategy.

4.6 Public dataset research

Historical transaction data can be studied through UCI's Online Retail dataset. Google publishes an obfuscated sample ecommerce BigQuery dataset useful for event-model exercises. Neither is a complete FD store data source, and neither automatically includes authentic marketing-to-order costs/attribution for this prototype. Details and direct links appear in REAL_WORLD_DATA_PLAN.md.

---

5. FEASIBILITY STUDY

5.1 Technical feasibility

The implemented system uses Python for processing; Streamlit for browser UI; Plotly for interactive charts; Pandas and NumPy for transformations; scikit-learn for optional clustering; MongoDB/pymongo for preferred data storage; and SQLite from Python standard library for offline demonstration. Required dependencies are declared in requirements.txt. A Windows batch launcher installs them in an isolated environment.

5.2 Economic feasibility

The offline demonstration can run on one personal computer without paying for hosting or a commercial database. Cloud MongoDB and hosting can involve account requirements, usage limits or charges; students must check current provider terms. No claim of permanent free cloud deployment is made.

5.3 Operational feasibility

A user can start the application from the Windows launcher, work with a pre-seeded dataset, filter visuals and download summarized reports. An authorized administrator can replace a dataset after schema validation and export a full backup. Faculty can inspect the active backend and audit records.

5.4 Schedule feasibility

The project can be completed incrementally: dataset design and validation → analytics formulas → UI → integration → testing → demonstration → final report. The full prior project was not thrown away conceptually; its intended analytical goals were retained while the implementation was rebuilt to address database and data-quality concerns.

5.5 Legal and privacy feasibility

Demonstration data avoid exposing actual customer records. Genuine imports require ownership or permission and data-minimization safeguards. Real customer data must not be uploaded to a publicly accessible instance without appropriate security review, lawful basis and institutional approval.

---

6. METHODOLOGY / PLANNING OF WORK

6.1 Requirements gathering

Functional requirements were based on the official project description and Phase 1 remarks: sales trends, customer preferences, ROI, dynamic data, persistent database integration and complete faculty demonstration. Additional system usability requirements included import/export, admin restrictions and accessible documentation.

6.2 Six-entity data model

Entity	Primary key	Main fields and relationships
Products	product_id	Name, category, unit price, cost snapshot, stock snapshot
Customers	customer_id	Customer label/name, signup date, region
Orders	order_id	Customer FK, date, status, payment method, total amount
Order Items	item_id	Order FK, product FK, quantity, selling price at purchase
Marketing Campaigns	campaign_id	Channel, campaign name, time window, spend, impressions, clicks
Campaign Conversions	attribution_id	Campaign FK, order FK, customer FK

The entity relationship diagram, architecture chart and Level 0/Level 1 data-flow charts are in ARCHITECTURE.md.

6.3 Generation and data provenance

The demonstration generator uses deterministic random seed 42. It creates India-themed fashion/accessory/home-decoration catalog entries, simulated Indian names and regions, dated customer registrations, seasonally weighted transaction activity, order statuses and compatible one-touch marketing attributions. This models plausible patterns but **does not observe or prove actual store behavior**. The seed is reproducible, letting faculty verify why results are stable. Data Studio displays the origin and import time.

6.4 Data validation and transformation

Input checks include: required columns; unique primary keys; foreign-key membership; one or more items per order; arithmetic agreement between order total and line items; permitted order statuses; dates and valid campaign windows; nonnegative monetary values; clicks no greater than impressions; valid order ownership for each attributed campaign; customer registration before order; and no attribution for cancelled/returned/shipped orders. The validator rejects invalid datasets before database activation.

6.5 Database integration

For MongoDB, a batch identifier is assigned to all six collections before an active metadata record is switched. The read path always uses the active batch. The old batch is deleted after the successful switch. The approach avoids dashboard reads of six collections in half-completed uploads; it is not equivalent to a multi-service enterprise distributed transaction. In SQLite mode, a transaction replaces all six SQL tables and writes metadata and an audit record.

6.6 Dashboard layout

The application contains eight navigable areas: Executive Overview; Sales Intelligence; Customer Intelligence; Campaign & ROI; Product Performance; Data Studio; Decision Lab; and Faculty Guide. Sales dashboards have common date, region and category filters. Marketing is calculated nationally rather than applying partial revenue filters without regional ad-spend allocation.

6.7 Sale recognition rule

Only orders with status **Delivered** count as realized revenue in this model. Orders marked Shipped are not yet delivered; Cancelled and Returned do not contribute realized revenue. This is a deliberately simplified managerial reporting rule, not a legally binding revenue recognition policy.

6.8 Marketing attribution rule

Campaign conversions are linked to one delivered order per event, with a maximum of one campaign relationship per order. During a user-selected partial campaign date range, marketing spend is allocated in proportion to overlapping campaign-days as an estimate. The method assumes uniform daily spend, which may not hold in practice; the interface says so explicitly.

6.9 Rule-based decision support

Recommendations are derived from actual active dataset rankings and segment counts rather than text invented by a generative model. The budget scenario multiplies a proposed budget by historic ROAS; it is labeled a purely arithmetic scenario and not represented as a reliable forecast.

6.10 Deployment and demonstration

Offline: Python virtual environment + bundled SQLite demonstration, started with RUN_WINDOWS.bat. Online/DB-first demonstration: MongoDB community service or Atlas connection from .env, seeded using python seed.py. Streamlit browser UI displays database mode; Mongo Compass independently verifies the saved collections.

---

7. KEY MATHEMATICAL METRICS

7.1 Sales indicators

Realized Revenue (₹) = sum of quantity × unit_price for order items belonging to **Delivered** orders.

Qualifying Orders = distinct delivered order_id values with at least one line included by currently selected filters.

Average Order Value (AOV) = Filtered Revenue / Qualifying Delivered Orders.

Total Units = sum of delivered line quantities in current scope.

Customers = number of unique purchasers linked to those qualifying sales lines.

Product Contribution Proxy = line revenue minus quantity multiplied by stored unit cost. This excludes logistics and overhead, and the cost column is an illustrative catalog snapshot.

7.2 Customer indicators

Recency (days) = (end-date + 1 day) − last completed purchase date.

Frequency = number of distinct delivered orders over each customer's available history up to the selected end date.

Monetary Value = total delivered revenue from that customer over the same history.

Repeat Buyer Share = customers with 2+ delivered orders / all customers with at least one delivered order × 100.

Cohort Retention for month k = customers whose first delivered purchase was cohort month m and who ordered again in month m+k / all acquired buyers in cohort m × 100. The cohort matrix shows *monthly repeat activity*, not consecutive retention or survival-adjusted rates.

7.3 Marketing indicators

Allocated Spend = full campaign spend × (overlapping days / total campaign days).

Attributed Revenue = sum of delivered order totals with a valid associated campaign conversion.

ROAS = Attributed Revenue / Allocated Spend (shown as a multiple).

CPA = Allocated Spend / Number of Attributed Delivered Orders. It is not customer acquisition cost unless all counted customers are verified new acquisitions.

Estimated Product Cost = sum of attributed-order item quantities × catalog unit cost.

Contribution ROI Proxy (%) = (Attributed Revenue − Estimated Product Cost − Allocated Spend) / Allocated Spend × 100.

CTR (%) = Estimated Clicks / Estimated Impressions × 100.

CVR (%) = Attributed Orders / Estimated Clicks × 100.

None of these metrics proves incremental profit or causal campaign effectiveness without additional cost and experimental data.

7.4 Worked ROAS example

Suppose valid campaign-attributed delivered order revenue is ₹75,000, allocated marketing spend is ₹25,000 and estimated product costs on those purchases are ₹40,000:

- ROAS = 75,000 / 25,000 = **3.00×**.
- Contribution after product cost = 75,000 − 40,000 = **₹35,000**.
- Contribution ROI proxy = (35,000 − 25,000) / 25,000 × 100 = **40%**.

These are demonstration arithmetic values, not claims about the delivered dataset.

---

8. FUNCTIONAL MODULES AND USER JOURNEY

8.1 Executive Overview

Five headline KPIs, monthly realized revenue area chart, product mix, strongest geographic sales, leading SKUs, and explainable suggestions. Changing filters recomputes the visual scope automatically.

8.2 Sales Intelligence

Status summary, monthly sales comparison, weekday order patterns, payment method composition, detailed order-level drill-through table, CSV export and Excel analysis export.

8.3 Customer Intelligence

Customer recency/frequency/spend distributions, labeled RFM segments, repeat purchase share, first-purchase cohort matrix, optional K-Means clusters, and segment export.

8.4 Campaign & ROI

National campaign totals, attributed sales, allocated advertising spend, ROAS, CPA, contribution ROI proxy, CTR, CVR, side-by-side channel comparisons and CSV/Excel export. A definitions panel explains the assumptions.

8.5 Product Performance

Product revenue, unit volumes, illustrative contribution margin, ABC revenue ranking and stock snapshots. Stock here is static demo information, not an actual order-driven inventory service.

8.6 Data Studio

Provenance status, record counts, database mode, audit history, table dictionary, data preview, protected Excel/CSV importer, synthetic reseeding and full backup ZIP.

8.7 Decision Lab

Evidence-based suggestions and a hypothetical channel budget comparison, clearly distinguished from a causal statistical model.

8.8 Faculty Guide

A seven-minute, reproducible faculty demo itinerary, definitions and direct mapping between official requirements and implementation screens.

---

9. TESTING AND QUALITY ASSURANCE

Tests are located in tests/test_system.py. Run python -m pytest tests -q from the project root. The prepared build ran 10 passing tests at package creation.

Test	Expected result
Deterministic seed	Same seed creates the same tables
Sales reconciliation	Item-level revenue equals delivered order total
Order status exclusion	No cancelled/returned/shipped revenue in realized sales
Category filtering	Revenue by categories sums to full revenue
Attribution aggregation	Conversions and channel totals match campaign totals
Campaign partial-period filter	Prorated spend does not exceed total campaign spend
Customer algorithms	Valid segments and 100% acquisition-month cohort
Invalid order total	Input is rejected before writing
Invalid customer attribution	Input is rejected
SQLite integration	Save, load, audit and failed-validation rollback

Test	Expected result
	behavior pass

Manual checks to show faculty: open the dashboard, change the region and dates, inspect numbers and charts, download CSV, log in as admin, preview Excel template, upload a valid full dataset, attempt a broken dataset to show rejection, and verify database mode in the sidebar.

10. FACILITIES REQUIRED FOR PROPOSED WORK

- A Windows laptop/desktop or macOS/Linux PC; 8 GB RAM recommended.
- Python 3.11/3.12, pip, web browser, and optionally VS Code and Git.
- MongoDB Community Server with Compass, or a MongoDB Atlas deployment for authentic MongoDB demonstration.
- Stable internet only for first-time dependency installation or hosted MongoDB. After setup, SQLite demo works offline.
- A source-data export or consented partner if the goal is to report authentic operational data. The seed data require no private external information.

11. EXPECTED OUTCOMES VS ACHIEVED OUTCOMES

Original synopsis goal	Implemented evidence
Central dashboard	Eight-page Streamlit app
Sales pattern visualization	Monthly, weekday, region, SKU and category charts
Customer preferences	RFM, cohorts, repeat share, payments and regions
Marketing ROI	Attributed ROAS, CPA, contribution ROI proxy and channel charts
Excel/CSV imports	Validated six-sheet or six-CSV importer
Persistent database	MongoDB or explicit SQLite demo selectable by config
Filter-driven interaction	Common date, region and product category filters; documented marketing scope
Business decisions	Data-derived strategies and bounded budget scenario
Exports and documentation	CSV/XLSX reports, full backups, audit records and faculty walkthrough
Live API integration	**Not implemented**; data persist and recalculate after manual imports
Genuine FD-store transactions	**Not supplied**; simulation shown honestly, with sourcing plan for future real data

12. LIMITATIONS, RECOMMENDATIONS AND FUTURE SCOPE

1. **Actual store feeds:** replace manual uploads with a permissioned Shopify/WooCommerce/order-platform API or webhook source with incremental sync and retry logic.
2. **Authentication:** replace single admin password with professional identity management and server-side role access before storing identifiable customer data.
3. **Better finance:** incorporate fulfillment, taxation, marketplace commission, refunds, shipping and true changing COGS before calling the output net profit.

4. **Marketing measurement:** obtain real UTM/CRM attributed conversions with documented attribution windows and campaign source costs; consider experiments for causal lift.
5. **Customer privacy:** use pseudonymized customer IDs, data minimization and approved storage retention.
6. **Segmentation quality:** monitor whether RFM clusters are stable and useful over time, rather than assuming algorithmic labels prove outcomes.
7. **Operational scaling:** use scheduled ETL, API authentication, streaming only when justified, indexes on date and customer IDs, and more robust tenancy and monitoring.
8. **Deployment:** secure managed hosting and automated CI/CD after privacy/access requirements are resolved. No hosted instance is supplied by this package.

---

13. FINAL CONCLUSION

The revised FD E-Commerce Analytics Dashboard fulfills the three central academic analytical themes: **sales trends**, **customer preferences** and **marketing ROI**, with integrated data, interactive visualization and concrete decision-support calculations. It improves substantially on the original fixed SQLite prototype by providing a MongoDB-capable persistence layer, auditable data ingestion, coherent sales/attribution validation, formal metric explanations, data export and comprehensive faculty demonstration material. The system is best described as a working, extensible **business analytics prototype**. Its present seed data are realistic simulations; authentic commercial conclusions require documented real data collection and permission.

---

14. REFERENCES

1. Streamlit documentation: https://docs.streamlit.io/
2. MongoDB official documentation: https://www.mongodb.com/docs/
3. PyMongo official documentation: https://pymongo.readthedocs.io/
4. Pandas documentation: https://pandas.pydata.org/docs/
5. Plotly Python documentation: https://plotly.com/python/
6. scikit-learn documentation: https://scikit-learn.org/stable/
7. Python documentation: https://docs.python.org/3/
8. UCI Online Retail dataset: https://archive.ics.uci.edu/dataset/352/online+retail
9. Google Analytics ecommerce BigQuery sample: https://developers.google.com/analytics/bigquery/web-ecommerce-demo-dataset
10. Provided six-page FD E-Commerce Analytics Dashboard Project Synopsis (original author: Jagjot Singh Passi).

DECLARATION FOR FACULTY

This report describes the rebuilt submitted implementation. Any future source-data substitution, external deployment or feature addition should be separately demonstrated and documented. All numerical findings from the bundled seed must be explicitly identified as **synthetic demonstration results**, not actual FD-store turnover or independently verified marketing returns.